CLASS ASSIGNMENT – 6
ID : 12341550
LRU

STEP 1
FILE : proc.h

```
struct frameinfo {
  uint va;
  pte_t *pte;
  int ref;
  uint last_used;
};

in struct proc
…
  struct frameinfo frames[16];
  int framecount;
```

STEP 2
FILE : vm.c

```
extern uint ticks;
...
```

modified allocum:

```
int
allocuvm(pde_t *pgdir, uint oldsz, uint newsz)
{
    char *mem;
    uint a;

    if (newsz >= KERNBASE)
        return 0;
```

```
    if (newsz < oldsz)
        return oldsz;

    a = PGROUNDUP(oldsz);
    for (; a < newsz; a += PGSIZE) {
        mem = kalloc();
        if (mem == 0) {
            cprintf("allocuvm out of memory\n");
            deallocuvm(pgdir, newsz, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if (mappages(pgdir, (char *)a, PGSIZE, V2P(mem),
PTE_W | PTE_U) < 0) {
            cprintf("allocuvm out of memory (2)\n");
            deallocuvm(pgdir, newsz, oldsz);
            kfree(mem);
            return 0;
        }

        // ---------------- LRU Update ----------------
        struct proc *curproc = myproc();
        if (curproc) {
            if (curproc->framecount < 16) {
                curproc->frames[curproc->framecount].va = a;
                curproc->frames[curproc->framecount].pte =
walkpgdir(pgdir, (void *)a, 0);
                curproc->frames[curproc->framecount].last_used
= ticks;
                curproc->framecount++;
            } else {
                // Select victim using LRU
                int victim_index = 0;
                for (int i = 1; i < curproc->framecount; i++) {
```

```
                    if (curproc->frames[i].last_used < curproc->frames[victim_index].last_used) {
                        victim_index = i;
                    }
                }

                // Free victim
                pte_t *vpte = curproc->frames[victim_index].pte;
                if (vpte && (*vpte & PTE_P)) {
                    uint vpa = PTE_ADDR(*vpte);
                    if (vpa != 0 && curproc->frames[victim_index].va < KERNBASE) {
                        kfree(P2V(vpa));
                        *vpte = 0;
                    }
                }

                // Replace with new
                curproc->frames[victim_index].va = a;
                curproc->frames[victim_index].pte = walkpgdir(pgdir, (void *)a, 0);
                curproc->frames[victim_index].last_used = ticks;
            }
        }
    }
    return newsz;
}

…
```

added helper function :

```
void update_lru_access(struct proc *p, uint va){
  for (int i = 0; i<p -> framecount; i++){
    if (p->frames[i].va == va)
```

```
      {
        p->frames[i].last_used = ticks; // update on access
        break;
      }
    }
  }
```

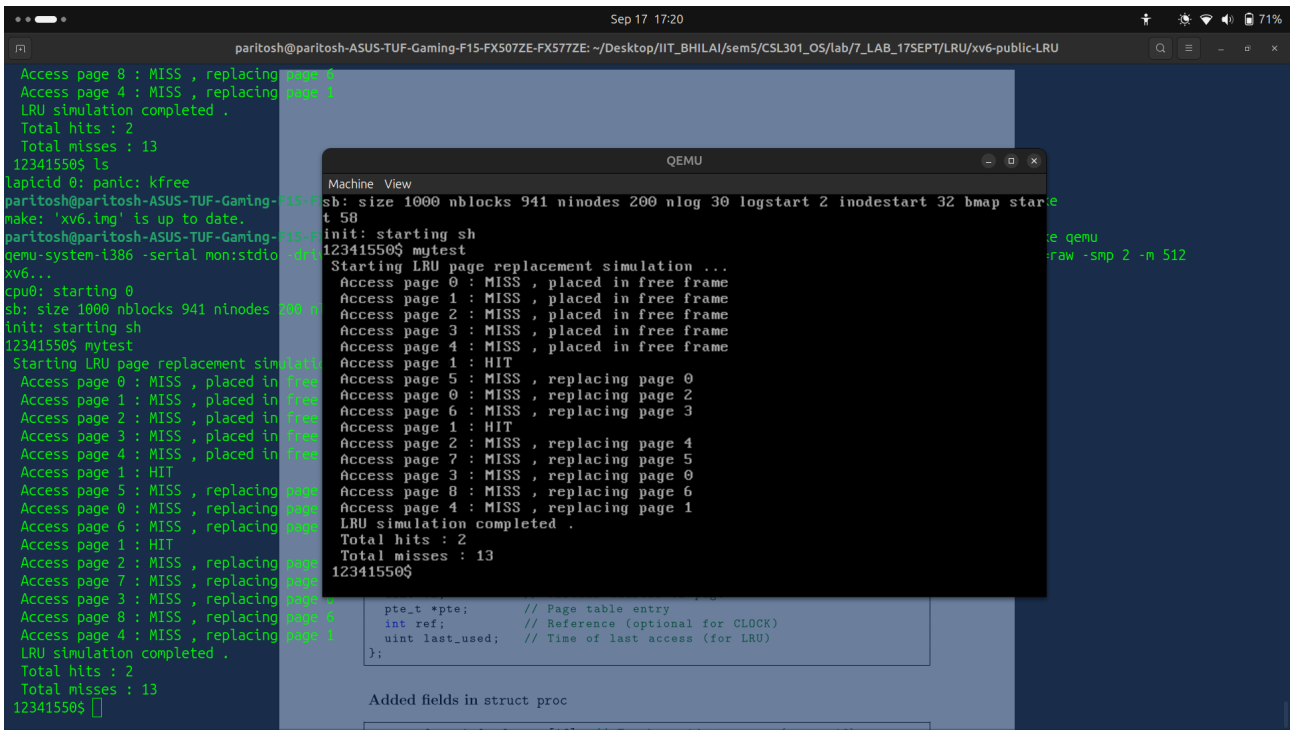
STEP 3
FILE : mytest.c

```
#include "types.h"
#include "user.h"
#define MAX_PAGES 5
#define TOTAL_ACCESSES 15
int main(int argc, char *argv[])
{
    int lru[MAX_PAGES];
    int last_used[MAX_PAGES];
    int i, j, page, hit = 0, miss = 0, time = 0;
    for (i = 0; i < MAX_PAGES; i++)
    {
        lru[i] = -1;
        last_used[i] = -1;
    }
    printf(1, " Starting LRU page replacement simulation ...\n
");
    int accesses[TOTAL_ACCESSES] = {0, 1, 2, 3, 4, 1, 5, 0,
6, 1, 2, 7, 3, 8, 4};
    for (i = 0; i < TOTAL_ACCESSES; i++)
    {
        page = accesses[i];
        time++;
        int found = 0;
        for (j = 0; j < MAX_PAGES; j++)
```

```
{
    if (lru[j] == page)
    {
        found = 1;
        last_used[j] = time;
        break;
    }
}
if (found)
{
    hit++;
    printf(1, " Access page %d : HIT \n ", page);
}
else
{
    miss++;
    int replaced = -1;
    for (j = 0; j < MAX_PAGES; j++)
    {
        if (lru[j] == -1)
        {
            lru[j] = page;
            last_used[j] = time;
            replaced = j;
            break;
        }
    }
    if (replaced == -1)
    {
        int lru_index = 0, min_time = last_used[0];
        for (j = 1; j < MAX_PAGES; j++)
        {
            if (last_used[j] < min_time)
            {
                min_time = last_used[j];
```

```
                    lru_index = j;
                }
            }
            printf(1, " Access page %d : MISS , replacing page
%d \n ", page,
                    lru[lru_index]);
            lru[lru_index] = page;
            last_used[lru_index] = time;
        }
        else
        {
            printf(1, " Access page %d : MISS , placed in free
frame \n ", page);
        }
    }
}
printf(1, " LRU simulation completed .\n ");
printf(1, " Total hits : %d \n ", hit);
printf(1, " Total misses : %d \n ", miss);
}
```

OUTPUT :


Sep 17 17:20
71%
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE: ~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/7_LAB_17SEPT/LRU/xv6-public-LRU
Access page 8 : MISS , replacing page 6
Access page 4 : MISS , replacing page 1
LRU simulation completed .
Total hits : 2
Total misses : 13
12341550$ ls
lapicid 0: panic: kfree
make: 'xv6.img' is up to date.
qemu-system-i386 -serial mon:stdio -drive
xv6...
cpu0: starting 0
init: starting sh
12341550$ mytest
Starting LRU page replacement simulation ...
Access page 0 : MISS , placed in free frame
Access page 1 : MISS , placed in free frame
Access page 2 : MISS , placed in free frame
Access page 3 : MISS , placed in free frame
Access page 4 : MISS , placed in free frame
Access page 1 : HIT
Access page 5 : MISS , replacing page 0
Access page 0 : MISS , replacing page 2
Access page 6 : MISS , replacing page 3
Access page 1 : HIT
Access page 2 : MISS , replacing page 4
Access page 7 : MISS , replacing page 5
Access page 3 : MISS , replacing page 0
Access page 8 : MISS , replacing page 6
Access page 4 : MISS , replacing page 1
LRU simulation completed .
Total hits : 2
Total misses : 13
12341550$
QEMU
Machine View
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
12341550$ mytest
Starting LRU page replacement simulation ...
Access page 0 : MISS , placed in free frame
Access page 1 : MISS , placed in free frame
Access page 2 : MISS , placed in free frame
Access page 3 : MISS , placed in free frame
Access page 4 : MISS , placed in free frame
Access page 1 : HIT
Access page 5 : MISS , replacing page 0
Access page 0 : MISS , replacing page 2
Access page 6 : MISS , replacing page 3
Access page 1 : HIT
Access page 2 : MISS , replacing page 4
Access page 7 : MISS , replacing page 5
Access page 3 : MISS , replacing page 0
Access page 8 : MISS , replacing page 6
Access page 4 : MISS , replacing page 1
LRU simulation completed .
Total hits : 2
Total misses : 13
12341550$
pte_t *pte; // Page table entry
int ref; // Reference (optional for CLOCK)
uint last_used; // Time of last access (for LRU)
};
Added fields in struct proc